NHS
Business Services Authority

Introduction to Linear Programming

Mihail Garbuzov

August 2026

NHS
Business Services Authority

Maximising the number of patients

Maximisation problem

Objective: maximise the number of patients treated per day

	Nurse	Doctor
Outpatient	45 min	1 h
Inpatient	3 h	30 min

The hospital has 25 doctors (200 h), 30 nurses (240 h) and 40 ward beds for inpatients.

A mathematical program

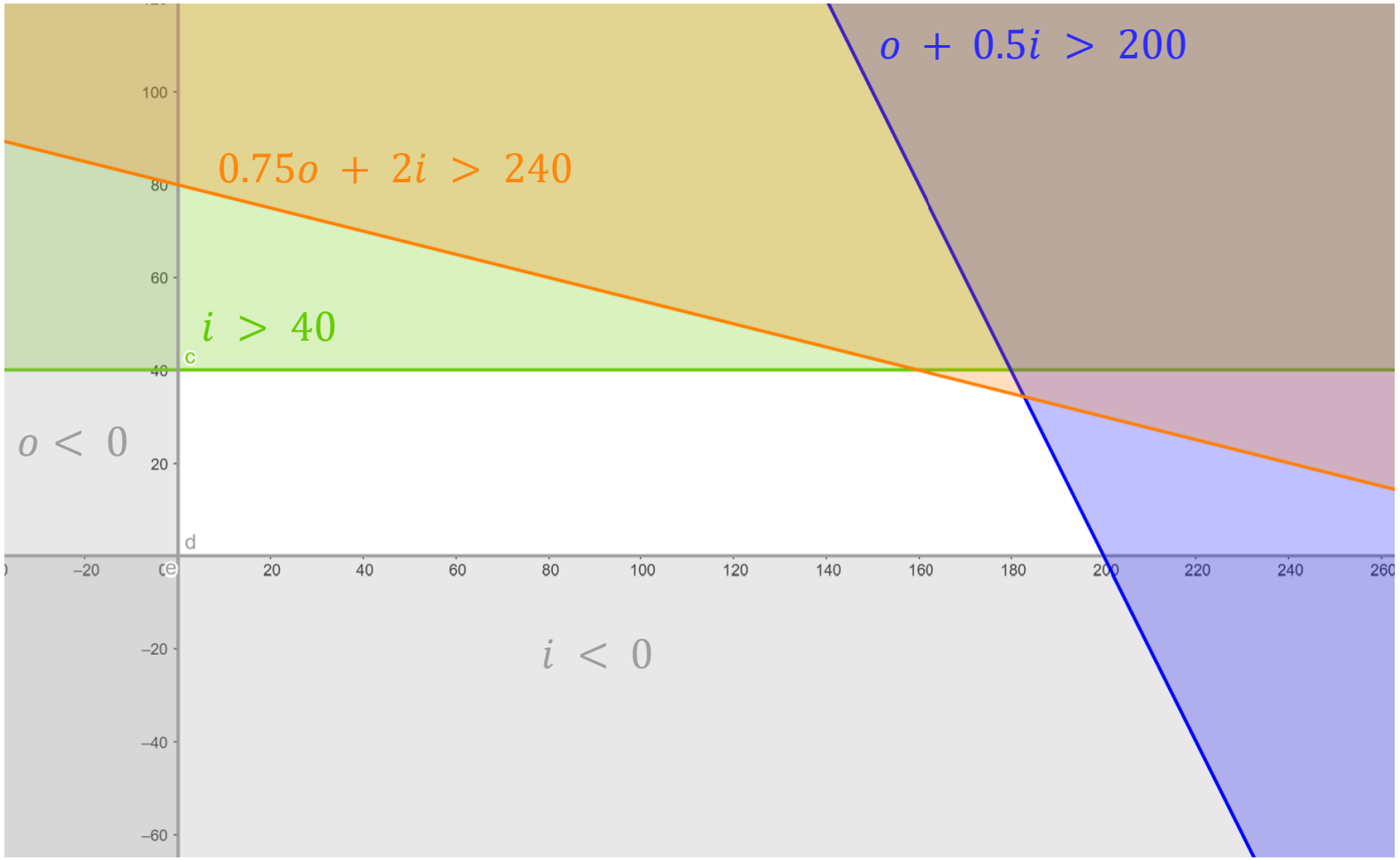
$$
\begin{array}{lll}
\text{Max} & P = o + i & \\
\text{Subject to} & 1o + 0.5i \leqslant 200 & \text{Doctor time constraint} \\
& 0.45o + 3i \leqslant 240 & \text{Nurse time constraint} \\
& i \leqslant 40 & \text{Ward bed constraint} \\
& o,\ i \geqslant 0 & \text{Non-negativity constraints}
\end{array}
$$

Maximisation problem

By convention, parts of inequalities that do **NOT** satisfy a solution are shaded. This makes it easier to visualize the feasible region, which remains unshaded.

The optimal solution is not at the vertex of a feasible region that is furthest is any one dimension, but one where the objective function has the highest value.

Graphic analysis is OK with 2 variables, difficult with 3, impossible with 4+



Graphic solution suggests ~217 patients

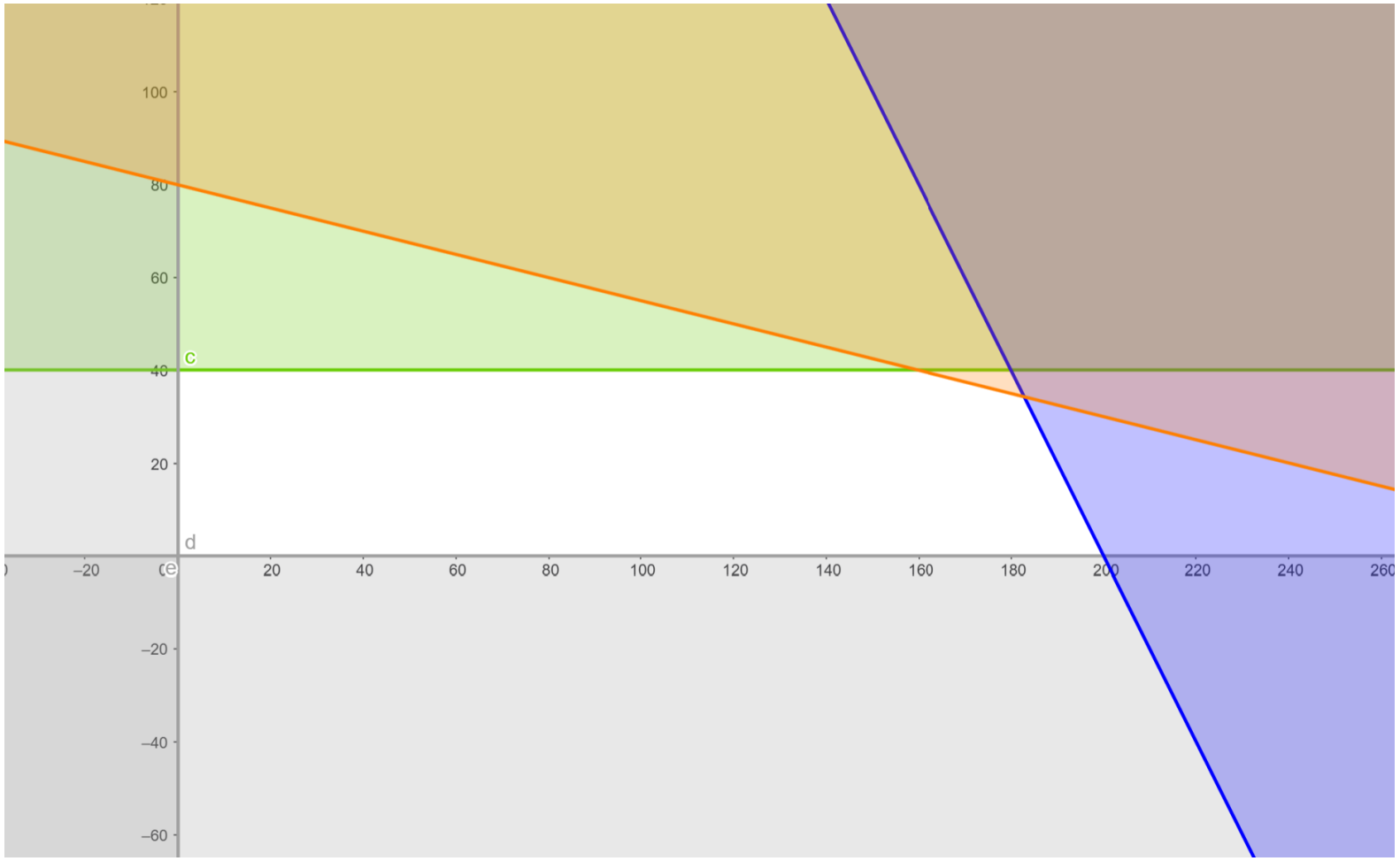
https://www.geogebra.org/calculator/kzvcv7pu

Solving LP problems with *n* variables: the Simplex algorithm

In LP, if a problem has an optimal solution, it is at a vertex of a feasible region. (Hence the name, the region in the vicinity of a vertex.)

With many variables and many constraints it is inefficient and sometimes practically impossible to check every vertex.

The algorithm starts from the Basic Feasible Solution (often the origin) and moves to the next connecting vertex which improves the objective function the most. When no improvement is possible, the optimal solution has been found.



R: lpSolve

$$\begin{aligned} \text{Max} \quad & P = o + i \\ \text{Subject to} \quad & 1o + 0.5i \leqslant 200 \\ & 0.45o + 3i \leqslant 240 \\ & i \leqslant 40 \\ & o,\ i \geqslant 0 \end{aligned}$$

```
library(lpSolve)

lp_object <- lp(
  direction = "max",

  # Coefficients of the objective function
  objective.in = c(1,1),

  # Coefficients of constraints
  const.mat = matrix(c(1,    0.5,
                       0.75, 3,
                       0,    1,
                       1,    0,
                       0,    1),
                     nrow = 5,
                     byrow = TRUE),

  # Signs of constraints
  const.dir = c("<=",
                "<=",
                "<=",
                ">=",
                ">="),

  # RHSs of constraints
  const.rhs = c(200,
                240,
                40,
                0,
                0),

  # Indices of variables are required to be integers
  int.vec = c(1,2)

)
```

```
> lp_object
Success: the objective function is 217
>
> lp_object$solution
[1] 183  34
```

NHS
Business Services Authority

Minimising staff costs

Minimisation problem

Objective: minimise daily staffing costs

	Day shift	Night shift
Regular nurse	£140	£200
Matron nurse	£240	£340
Minimum staffing	20 nurses	12 nurses
Minimum supervision ratio	15%	
Minimum collective regular nurse shifts (trade union agreement)	30 (across 24 h)	

$$\text{Min} \quad P = 140r_d + 240m_d + 200r_n + 340m_n$$

Subject to	$r_d + m_d \geqslant 20$	Day minimum staffing
	$r_n + m_n \geqslant 12$	Night minimum staffing
	$m_d \geqslant 0.15(r_d + m_d)$	Day minimum supervision ratio
	$m_n \geqslant 0.15(r_n + m_n)$	Night minimum supervision ratio
	$r_d + r_n \geqslant 20$	Collective regular nurse shifts
	$r_d,\ m_d,\ r_n,\ m_n \geqslant 0$	Non-negativity

$$0.85m_d - 0.15r_d \geqslant 0$$
$$0.85m_n - 0.15r_n \geqslant 0$$

This problem could be further complicated by adding

- *More nurse types (paediatric, geriatric, oncology, operating theater etc.) and minimum staffing by nurse/shift type*
- *Variable demand by day of the week and time of day (e.g. Tuesday afternoon vs Friday night)*
- *Overlapping shifts*

R: lpSolve

$$\text{Min} \quad P = 140r_d + 240m_d + 200r_n + 340m_n$$

$$\text{Subject to} \quad r_d + m_d \geqslant 20$$
$$r_n + m_n \geqslant 12$$
$$0.85m_d - 0.15r_d \geqslant 0$$
$$0.85m_n - 0.15r_n \geqslant 0$$
$$r_d + r_n \geqslant 30$$
$$r_d,\ m_d,\ r_n,\ m_n \geqslant 0$$

```
library(lpSolve)

lp_object <- lp(
  direction = "min",

  # Coefficients of the objective function
  objective.in = c(140,240,200,340),

  # Coefficients of constraints
  const.mat = matrix(c(1,    1,     0,    0,
                       0,    0,     1,    1,
                    -0.15, 0.85,    0,    0,
                       0,    0,  -0.15, 0.85,
                       1,    0,     1,    0,
                       1,    0,     0,    0,
                       0,    1,     0,    0,
                       0,    0,     1,    0,
                       0,    0,     0,    1),
                     nrow = 9,
                     byrow = TRUE),

  # Signs of constraints
  const.dir = rep(">=", 9),

  # RHSs of constraints
  const.rhs = c(20,
                12,
                0,
                0,
                30,
                0,
                0,
                0,
                0),

  # Indices of variables are required to be integers
  int.vec = c(1,2,3,4)

)
```

```
> lp_object
Success: the objective function is 6440
>
> lp_object$solution
[1] 20  4 10  2
```

NHS
Business Services Authority

Applications in Game Theory

Game Theory

Game theory is a mathematical framework for analysing interactions between rational players.

This game is zero-sum, since the gain of one players is exactly the loss of another player.

The optimal strategy is to try to play safe. The best of the worst payoffs are:

- From CFO's perspective (maximin): auditing (-1)
- From department's perspective (minimax): honest request (0).

If both adopt pure play safe strategies, a CFO a will have an incentive to forego the auditing, in which case the department will have an incentive to punish for it by padding their request.

Therefore, the game has no stable pure strategy.

	Department submits an honest lean request	**Department submits an aggressive padded request**
CFO trusts and fast-tracks	0 *(efficient baseline)*	-5 *(CFO approves budget fat)*
CFO audits	-1 *(CFO wastes resource on an audit, department gains political points for being wrongly audited)*	2 *(CFO catches padding and cuts budget as a penalty)*

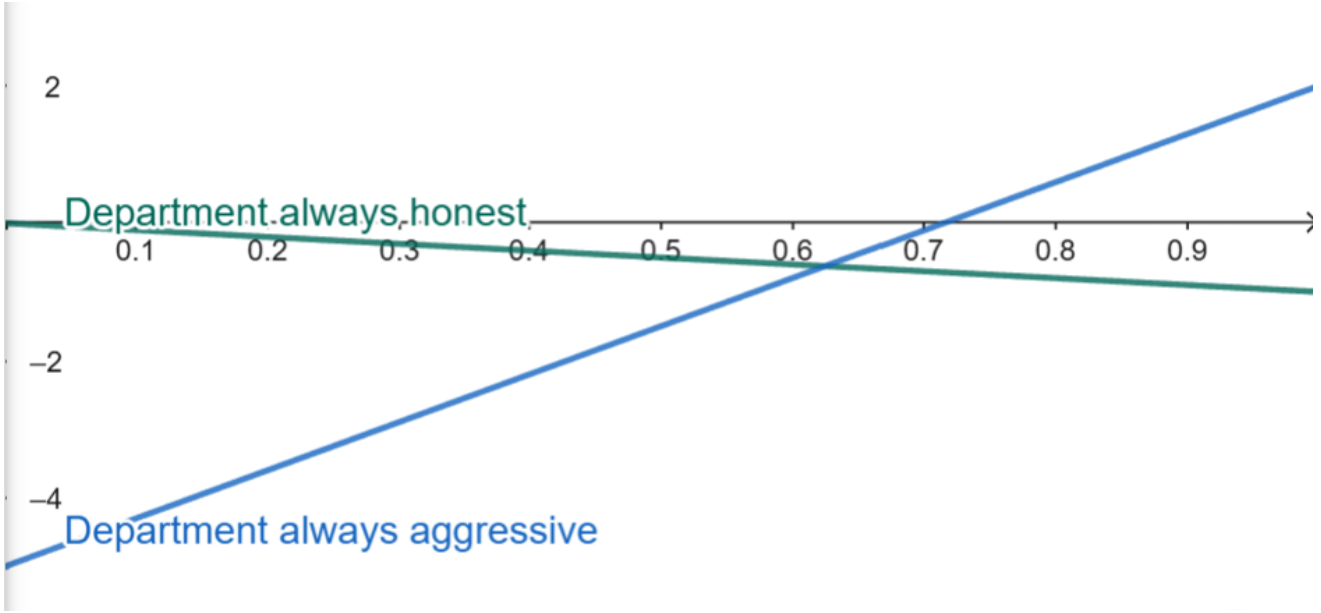
Mixed Strategies

If CFO adopts a mixed strategy, where they audit with probably *p* and trust with probability *(1-p)*.

	Department: honest	Department: aggressive
CFO: trust	0	-5
CFO: audit	-1	2

The pay-off to the CFO over the whole range of *p* is

- If department is always honest: $-p + 0(1-p) = -p$
- If department is always aggressive: $2p - 5(1-p) = 7p - 5$



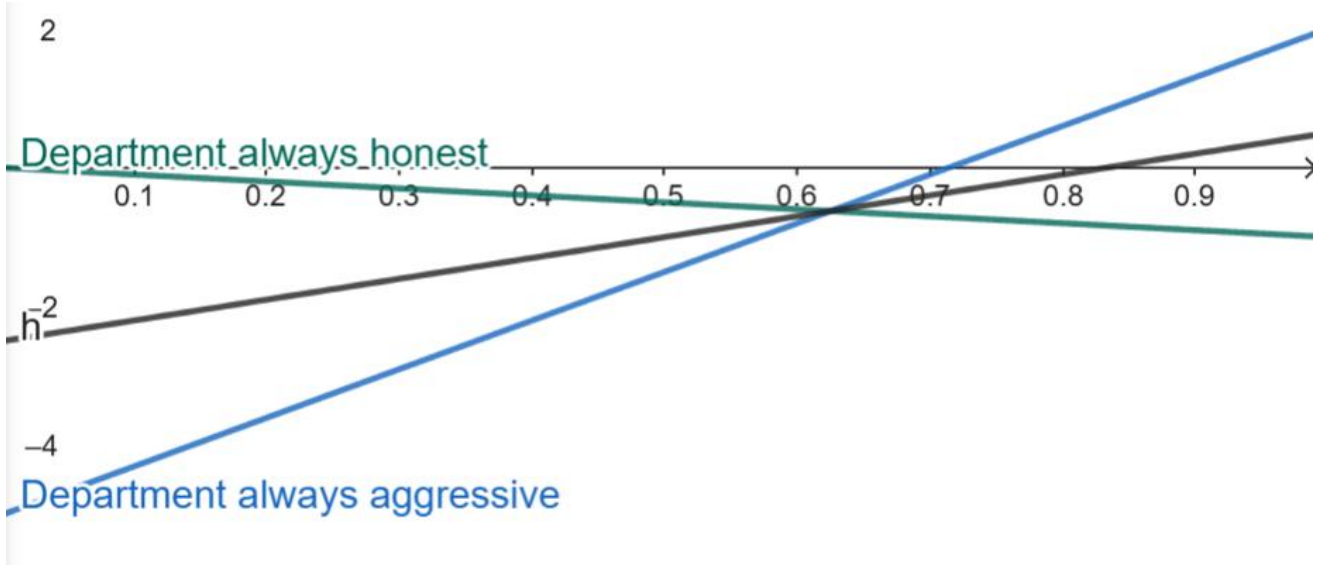
CFO's safe strategy corresponds to the best of the worst payoffs (maximin) across the possible strategies of the department. Here, at $p \approx 0.63$.

Mixed Strategies

The pay-off to the CFO over the whole range of *p* is

- If department is always honest: $-p + 0(1-p) = -p$
- If department is always aggressive: $2p - 5(1-p) = 7p - 5$
- If department is honest with probability *q* and aggressive with probability *(1-q)*:

$$-pq + (7p - 5)(1 - q)$$



https://www.geogebra.org/calculator/fgabnqq5

	Department: honest	Department: aggressive
CFO: trust	0	-5
CFO: audit	-1	2

If the department also adopts a mixed strategy, this would not change the optimal strategy of the CFO.

In fact, the CFO could reveal their strategy to the department and the department would still not be able to improve!

This is one of the most famous results in Game Theory (John von Neumann's Minimax theorem 1928).

Mixed Strategies: more options

As players have more options, it becomes impossible to use the graphic method and the algebraic method becomes very long winded with increasingly complicated systems of equations.

	Department: honest	**Department: aggressive**	**Department: mild padding**
CFO: trust	0	-5	-2.5 *(moderate win for the department)*
CFO: audit	-1	2	1 *(moderate penalty)*
CFO: AI screen	1.5 *(AI may slightly further improve efficiency of an honest request)*	-2 *(AI finds some padding in aggressive requests)*	-1 *(AI finds less padding)*

Game Theory: LP formulation

The most elegant approach is to formulate and solve the problem as a linear program.

For a technical reason, the Simplex algorithm requires non-negative pay-off coefficients, so we add at least 5 to each.

This changes the optimal value of the game by the same amount, but doesn't change the optimal strategy (because relative distances between payoffs are still the same).

	Department: honest	Department: aggressive	Department: mild padding
CFO: trust	0	-5	-2.5
CFO: audit	-1	2	1
CFO: AI screen	1.5	-2	-1

Add 5

	Department: honest	Department: aggressive	Department: mild padding
CFO: trust	5	0	2.5
CFO: audit	4	7	6
CFO: AI screen	6.5	3	4

Max	$P = V$	Value of the game to the CFO (can be thought of as $0p_1 + 0p_2 + 0p_3 + V$)
Subject to	$5p_1 + 4p_2 + 6.5p_3 \geqslant V$	If the department is always honest
	$0p_1 + 7p_2 + 3p_3 \geqslant V$	If the department is always aggressive
	$2.5p_1 + 6p_2 + 4p_3 \geqslant V$	If the department is always mildly padding
	$p_1 + p_2 + p_3 = 1$	
	$p_1,\ p_2,\ p_3 \geqslant 0$	Non-negativity

Game Theory: LP formulation in R

$$\text{Max} \quad P = V$$

$$\begin{aligned}
\text{Subject to} \quad & 5p_1 + 4p_2 + 6.5p_3 \geqslant V \\
& 0p_1 + 7p_2 + 3p_3 \geqslant V \\
& 2.5p_1 + 6p_2 + 4p_3 \geqslant V \\
& p_1 + p_2 + p_3 = 1 \\
& p_1,\ p_2,\ p_3 \geqslant 0
\end{aligned}$$

```
library(lpSolve)

lp_object <- lp(
  direction = "max",

  # Coefficients of the objective function
  objective.in = c(0,0,0,1),

  # Coefficients of constraints
  const.mat = matrix(c(5,   4,  6.5, -1,
                       0,   7,  3,   -1,
                       2.5, 6,  4,   -1,
                       1,   1,  1,    0,
                       1,   0,  0,    0,
                       0,   1,  0,    0,
                       0,   0,  1,    0),
                     nrow = 7,
                     byrow = TRUE),

  # Signs of constraints
  const.dir = c(">=",
                ">=",
                ">=",
                "=",
                ">=",
                ">=",
                ">="),

  # RHSs of constraints
  const.rhs = c(0,
                0,
                0,
                1,
                0,
                0,
                0)

)
```

```
> # Value of the game (adjusted backwards)
> lp_object$objval - 5
[1] 0.1111111
>
> # Option probabilities
> lp_object$solution[1:3]
[1] 0.0000000 0.5555556 0.4444444
```

Given the 3 options available to each player, the optimal strategy for the CFO is to

- Never trust
- Full audit with probability 56%
- AI screen with probability 44%